

Конструктор из геометрических примитивов

Группа: ИУ7-53Б

Студент: Семенчук М. А.

Руководитель: Шибанова Д. А.

Цели и задачи

Целью данной работы является разработка ПО для визуализации трехмерных сцен, состоящих из геометрических примитивов, а также проведение исследования по оценке производительности разработанного ПО.

Задачи:

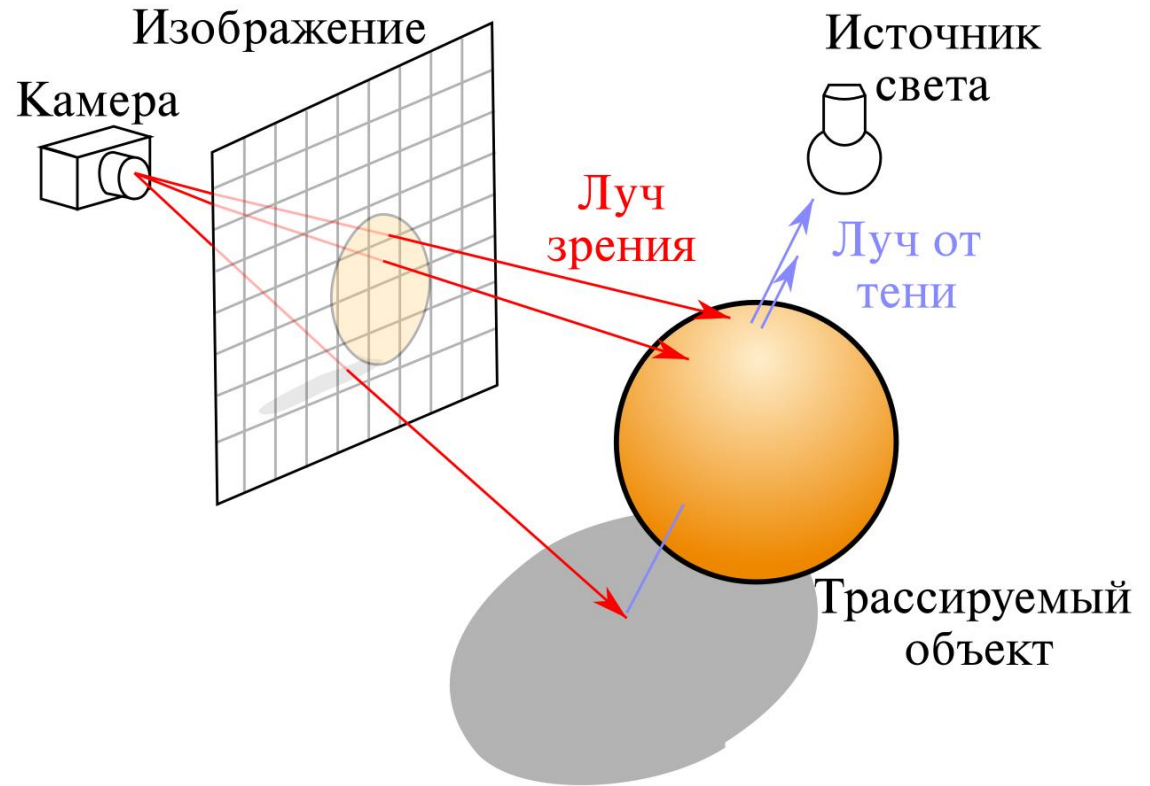
- выбрать алгоритм генерации изображения, модель освещения и затенения;
- описать доступные для размещения на сцене модели и формализовать их;
- построить схемы алгоритмов;
- выбрать средства реализации (платформу, язык программирования, графическую библиотеку, GUI библиотеку);
- реализовать спроектированное ПО с помощью выбранных средств;
- провести исследование производительности разработанного ПО.

Трассировка лучей

Трассировка лучей - метод рендеринга, который позволяет реалистично имитировать освещение среды за счет использования физически корректной модели поведения света.

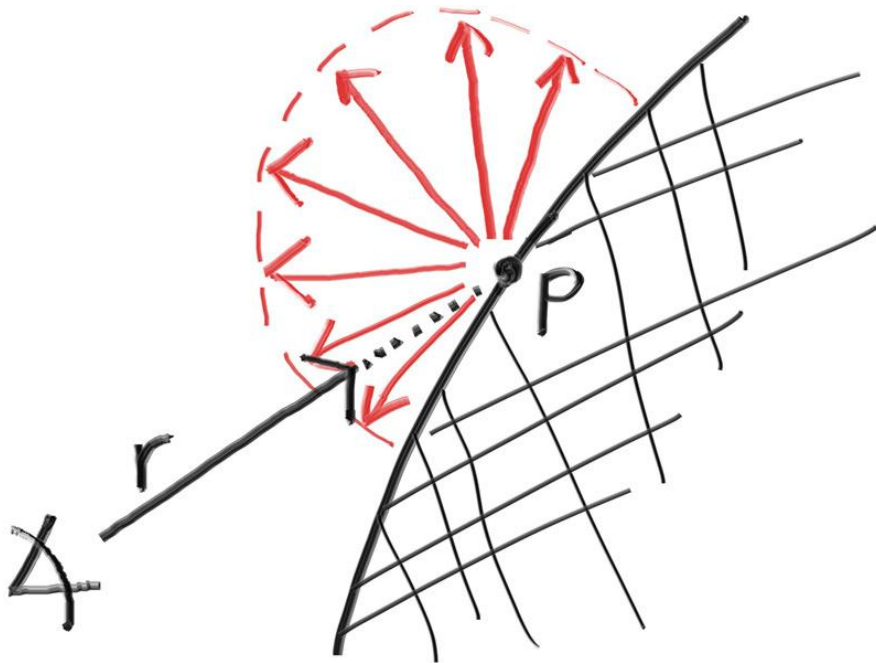
Распространяясь по сцене и взаимодействуя с ее объектами, свет может:

- Отражаться;
- Преломляться;
- Поглощаться;
- рассеиваться.

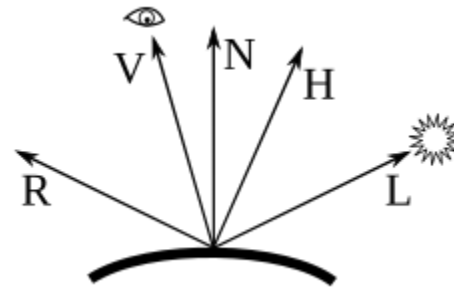
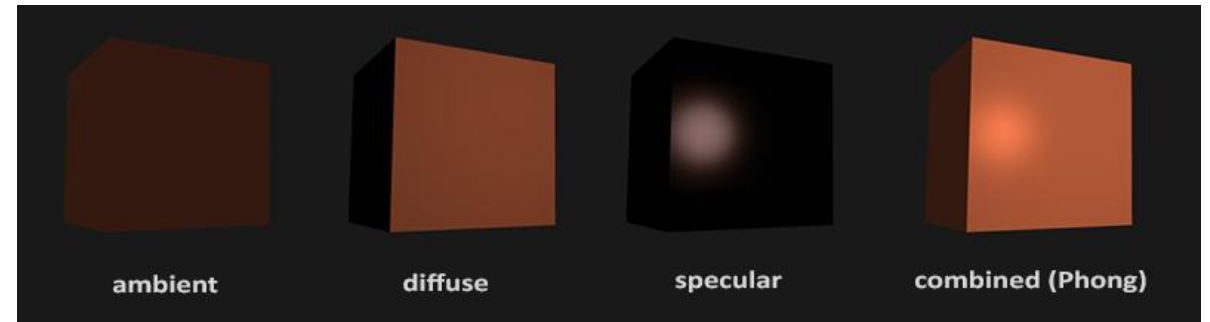


Модели освещения

Ламберт



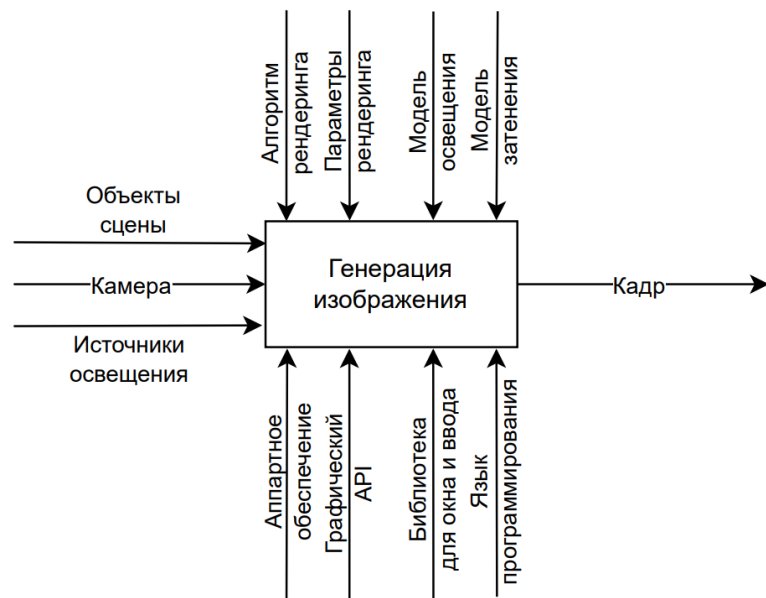
Фонг



V – наблюдатель
 N – нормаль
 L – источник света
 R – отраженный свет
 H – полувектор

Формализованная постановка задачи

IDEF0 диаграмма
обобщенного алгоритма
генерации изображения



IDEF0 диаграмма
выбранного алгоритма
генерации изображения

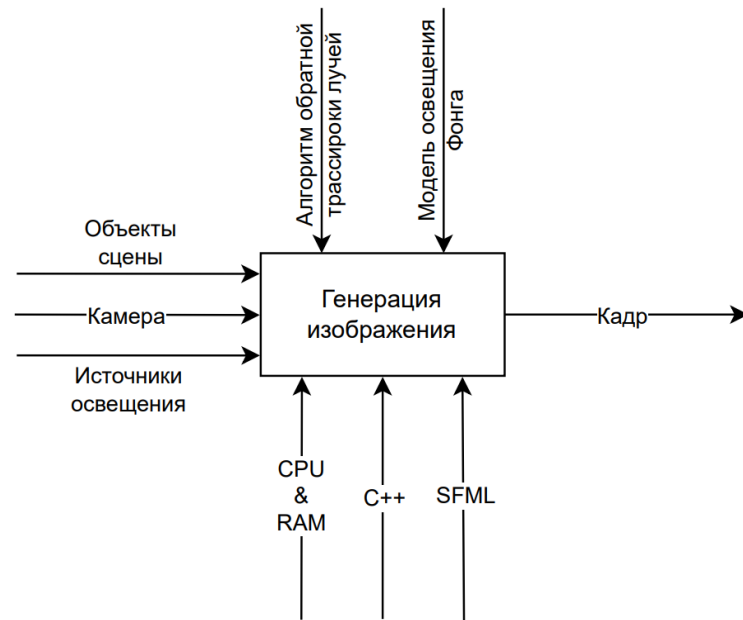
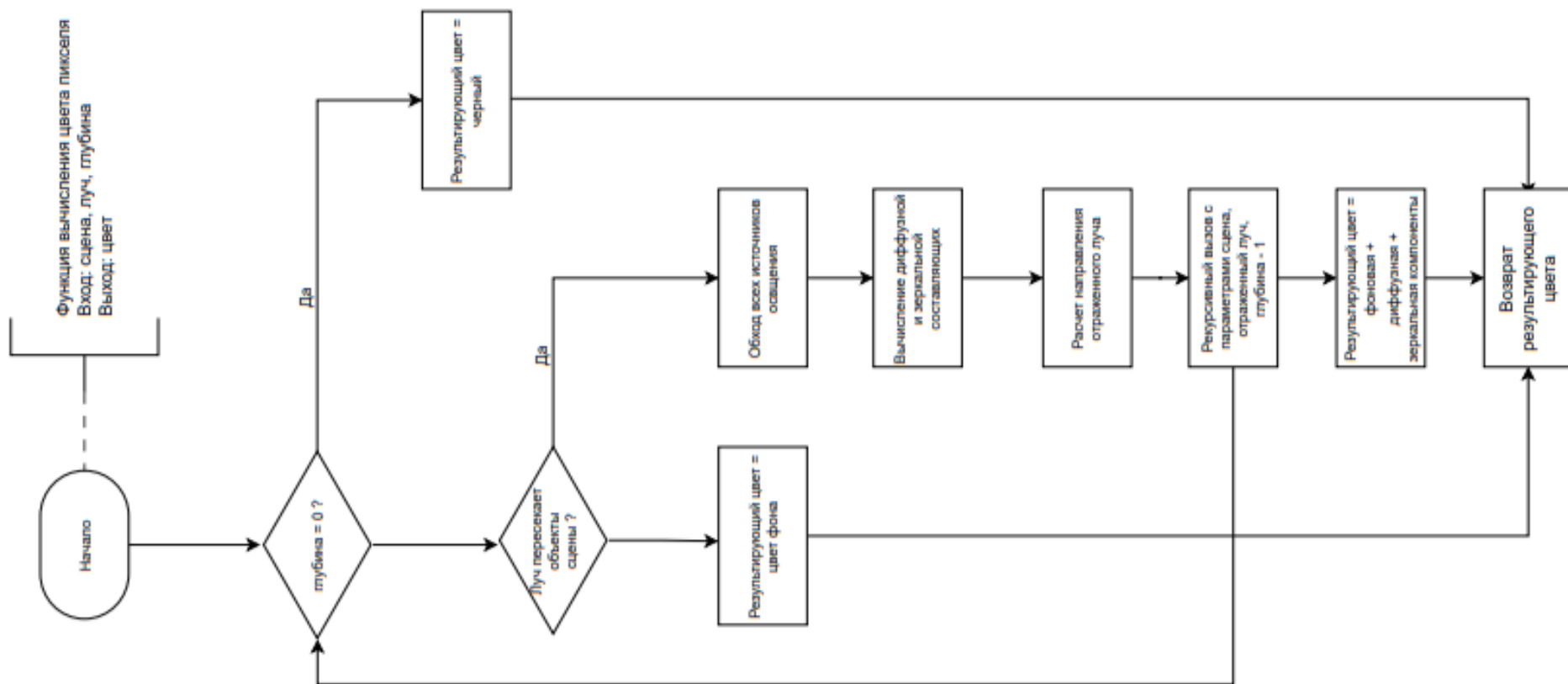


Схема алгоритма трассировки лучей



Требования к ПО и средства реализации

Разрабатываемое ПО должно обладать следующим функционалом:

- добавление и удаление объектов;
- изменение положения, ориентации, масштаба и цвета объектов;
- изменение положения и ориентации камеры;
- добавление и удаление источников освещения;
- изменение положения, ориентации и цвета источников освещения.

Средства реализации:

- Язык программирования: C++
- Графическая библиотека: SFML
- GUI библиотека: Dear ImGui

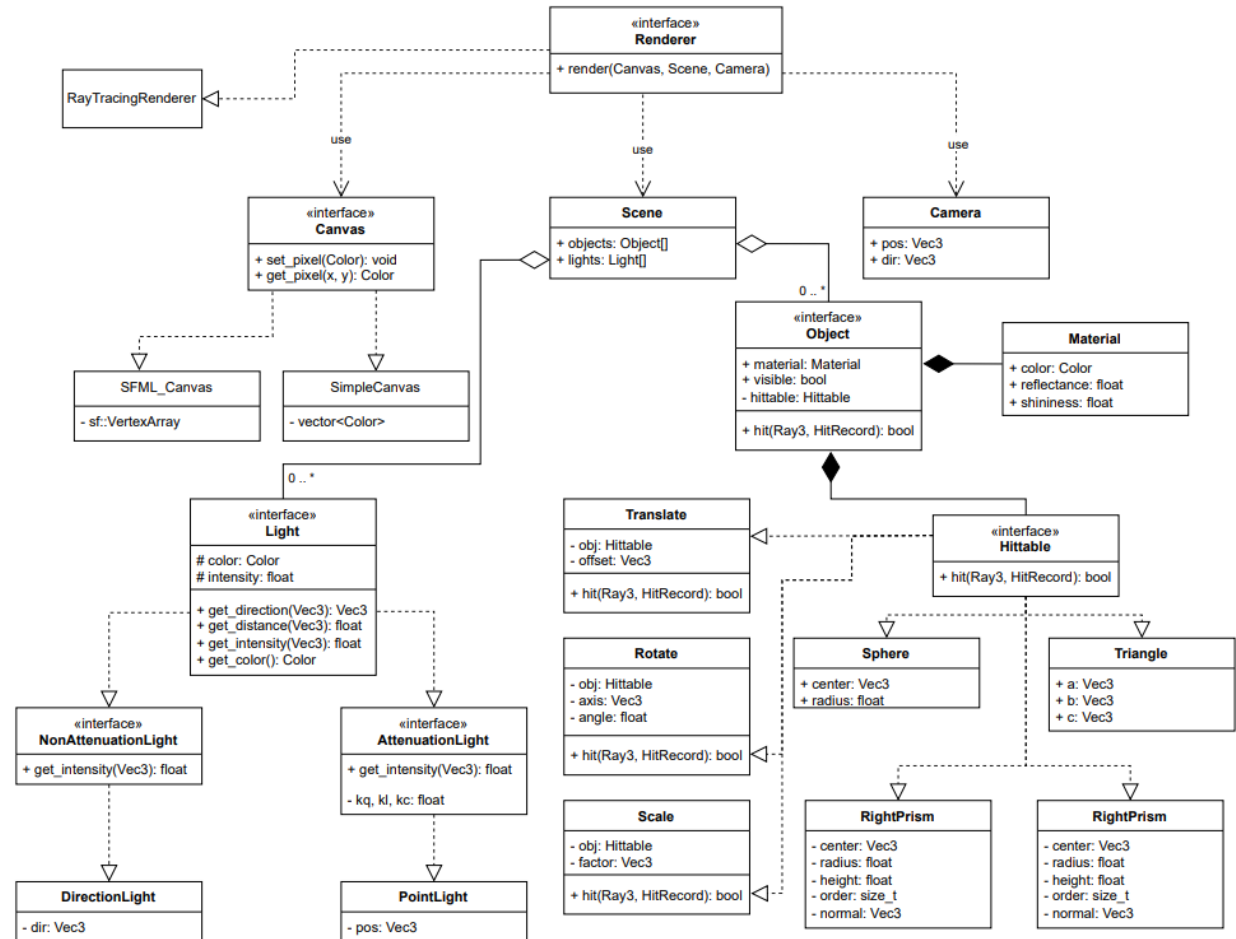
Диаграмма классов

Класс **Scene** предназначен для хранения геометрических объектов сцены и источников освещения.

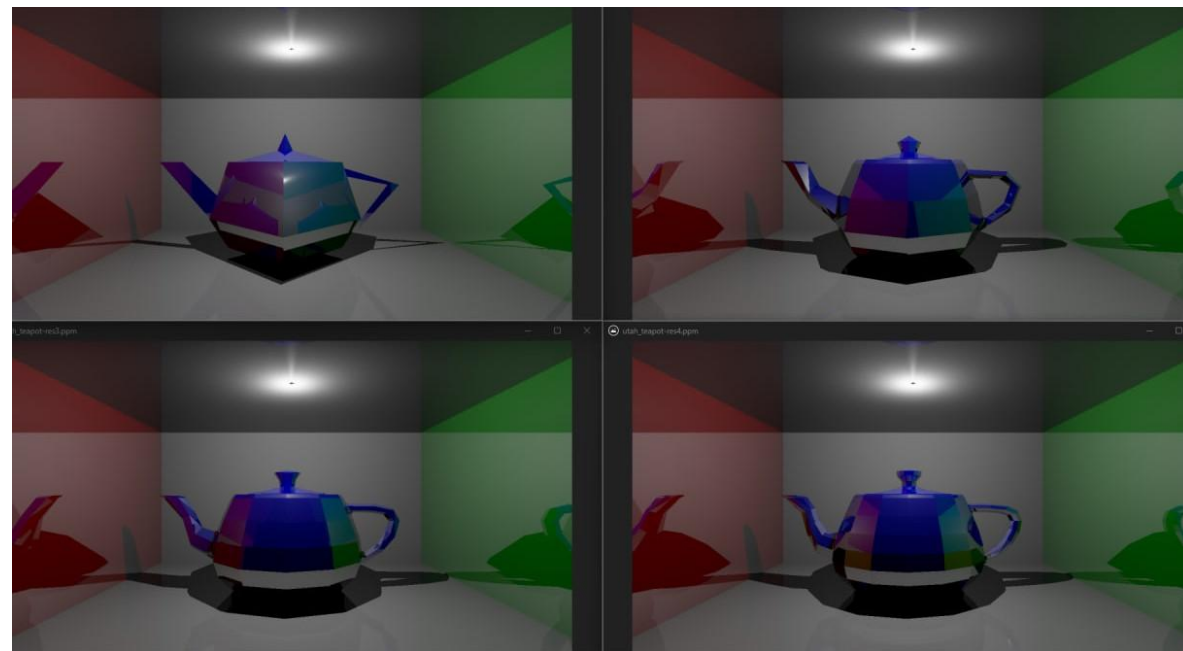
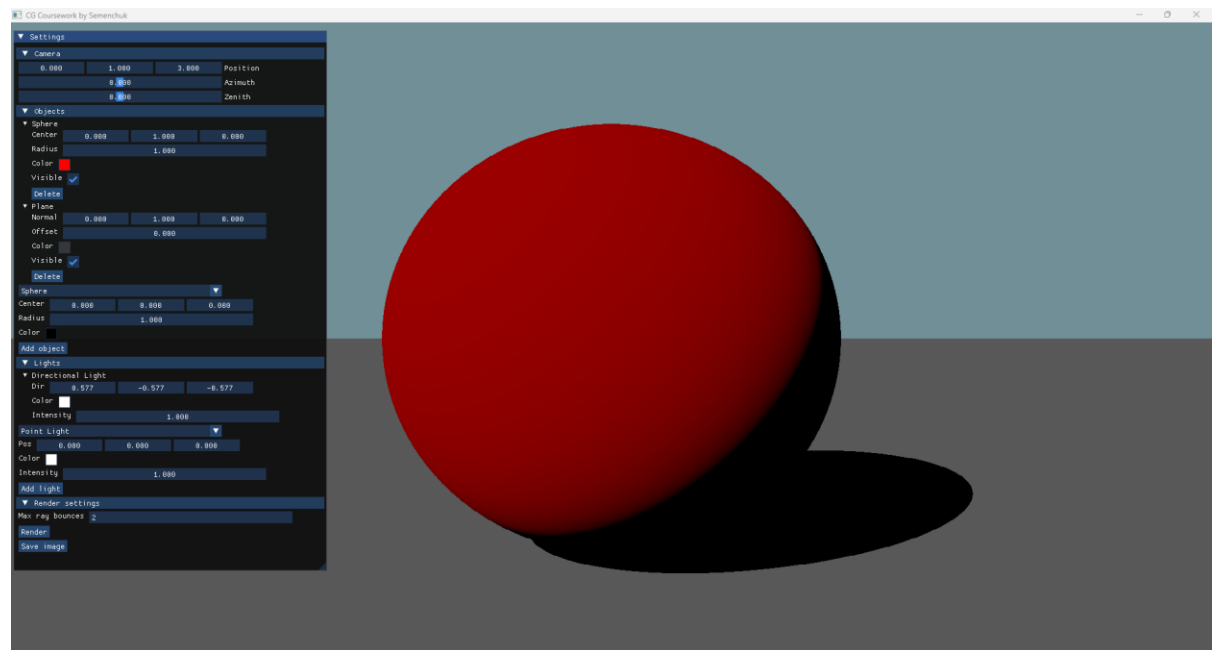
Класс **Camera** инкапсулирует логику работы камеры, включая ее положение и ориентацию.

Класс **Canvas** используется для сохранения сгенерированного изображения.

Класс **Renderer** осуществляет формирование изображения сцены (Scene) с позиции камеры (Camera) и записывает результат в выходной буфер (Canvas). Этот класс выполняет роль центрального модуля, обеспечивая взаимодействия всех остальных компонентов системы.



Интерфейс и пример работы ПО



Чайник университета Юта
с различным числом полигонов (100, 500, 1000, 2000)
в Корнеллской коробке

Исследование времени генерации кадра

Тестовая сцена состоит из:

- Корнеллской коробки;
- чайника Юта;
- точечного источника освещения.

Параметра генерации кадра:

- разрешение кадра 1280x720 пикселей;
- максимальное число переотражений луча = 2.

Параметры системы,
на которой будут проводиться замеры времени:

- CPU - 6C/12T @ 3.7GHz;
- RAM - 32GB DDR5;
- OS - Windows 11.

Способ распараллеливания – стандарт OpenMP,
реализованный в языке C++ через директиву
#pragma.

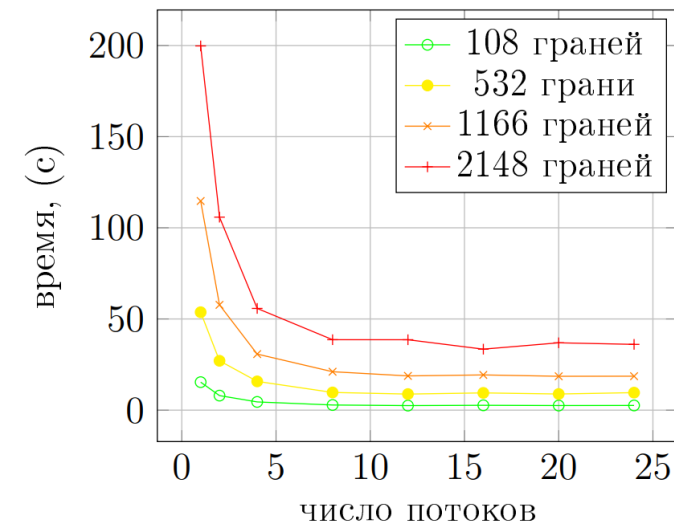
Для исследования времени генерации
кадра были выбраны следующие
варианты числа потоков:

- Однопоточный режим (1);
- Количество потоков, равное числу физических ядер процессора (6);
- Количество потоков, равное числу логических ядер процессора (12);
- Вдвое большее количество потоков по сравнению с числом логических ядер процессора (24).

Исследование времени генерации кадра

Число граней	1 поток	6 потоков	12 потоков	24 потока
108	15.4	4.5	2.5	2.6
532	53.7	15.8	8.8	9.6
1166	114.7	30.7	18.8	18.6
2148	199.8	55.8	38.6	36.1

Анализ представленных графиков позволяет сделать вывод, что прирост производительности с увеличением числа потоков носит ограниченный характер. Наиболее эффективным числом потоков является количество логических ядер процессора, при котором достигается максимальная параллельная загрузка. При дальнейшем увеличении числа потоков наблюдается снижение производительности, что связано с дополнительными затратами процессорного времени на переключение контекста между потоками.



Выводы

Цель работы достигнута – разработано ПО для визуализации трехмерных сцен, состоящих из геометрических примитивов.

В ходе выполнения работы были последовательно выполнены поставленные задачи:

1. выбрать алгоритм генерации изображения, модель освещения и затенения;
2. описаны доступные для размещения на сцене модели и формализовать их;
3. построены схемы алгоритмов;
4. выбраны средства реализации;
5. реализовано спроектированное ПО с помощью выбранных средств;
6. проведено исследование производительности разработанного ПО.